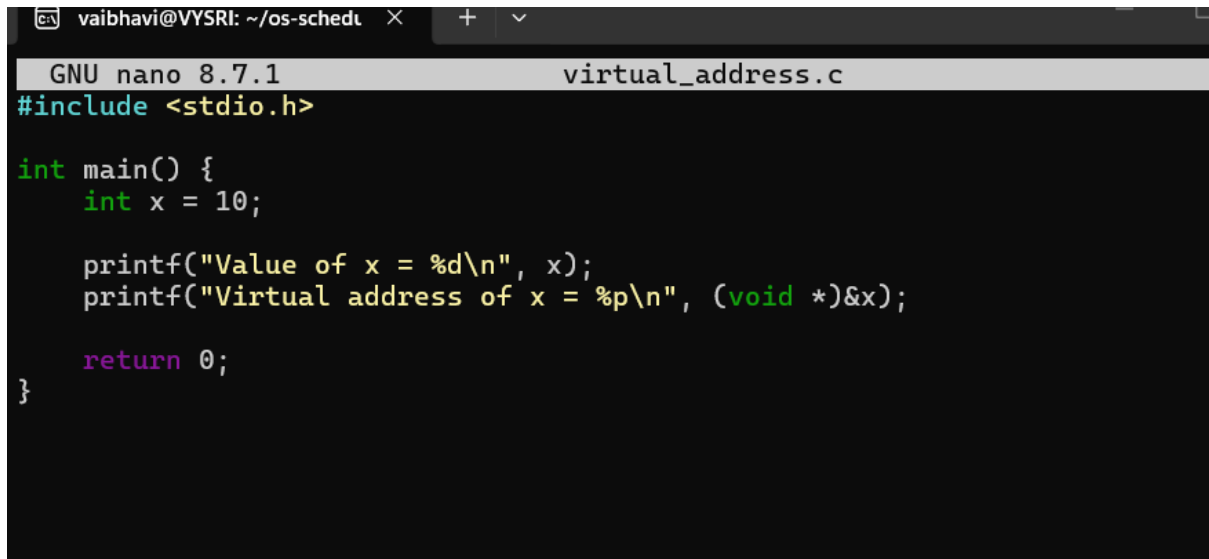


Name: S. Vaibhava Sree

Roll No: 2520030398

Subject: OSSP(PRACTICAL)

1.Virtual Address:

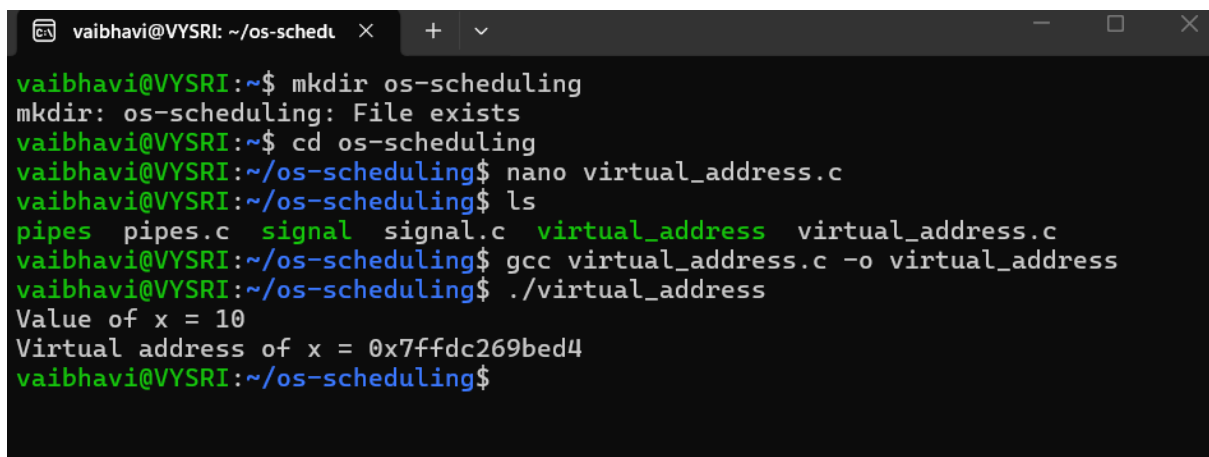


```
GNU nano 8.7.1 virtual_address.c
#include <stdio.h>

int main() {
    int x = 10;

    printf("Value of x = %d\n", x);
    printf("Virtual address of x = %p\n", (void *)&x);

    return 0;
}
```



```
vaibhavi@VYSRI: ~/os-schedt
vaibhavi@VYSRI:~$ mkdir os-scheduling
mkdir: os-scheduling: File exists
vaibhavi@VYSRI:~$ cd os-scheduling
vaibhavi@VYSRI:~/os-scheduling$ nano virtual_address.c
vaibhavi@VYSRI:~/os-scheduling$ ls
pipes  pipes.c  signal  signal.c  virtual_address  virtual_address.c
vaibhavi@VYSRI:~/os-scheduling$ gcc virtual_address.c -o virtual_address
vaibhavi@VYSRI:~/os-scheduling$ ./virtual_address
Value of x = 10
Virtual address of x = 0x7ffdc269bed4
vaibhavi@VYSRI:~/os-scheduling$
```

2. Process Spaces:

```
vaibhavi@VYSRI: ~/os-schedul  X + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>

int global = 10;
int bss_variable;

int main() {

    int local = 20;

    int *heap = malloc(sizeof(int));
    *heap = 30;

    printf("Code address   : %p\n", (void *)main);
    printf("Data address   : %p\n", (void *)&global);
    printf("BSS address    : %p\n", (void *)&bss_variable);
    printf("Heap address   : %p\n", (void *)heap);
    printf("Stack address  : %p\n", (void *)&local);

    free(heap);

    return 0;
}

Stack address : 0x7fffc851ba5ac
vaibhavi@VYSRI:~/os-scheduling$ cd ~/os-scheduling
vaibhavi@VYSRI:~/os-scheduling$ nano memory.c
vaibhavi@VYSRI:~/os-scheduling$ ls
memory  memory.c  pipes  pipes.c  signal  signal.c  virtual_address  virtual_address.c
vaibhavi@VYSRI:~/os-scheduling$ gcc memory.c -o memory
vaibhavi@VYSRI:~/os-scheduling$ ./memory
Code address   : 0x5f66797921a9
Data address   : 0x5f6679795010
BSS address    : 0x5f6679795018
Heap address   : 0x5f66b8d55010
Stack address  : 0x7ffc851ba5ac
vaibhavi@VYSRI:~/os-scheduling$
```

3. Demand Pages:

```

GNU nano 8.7.1
#include <stdio.h>

int main() {

    int page_table[4] = {1, -1, 3, -1};

    int page;

    printf("Enter page number (0-3): ");
    scanf("%d", &page);

    if (page_table[page] == -1) {
        printf("Page Fault!\n");
        printf("Page must be loaded into memory.\n");
    }
    else {
        printf("Page is already in memory.\n");
        printf("Physical frame = %d\n", page_table[page]);
    }

    return 0;
}

```

```

vaibhavi@VYSRI:~/os-scheduling$ cd ~/os-scheduling
vaibhavi@VYSRI:~/os-scheduling$ nano paging.c
vaibhavi@VYSRI:~/os-scheduling$ gcc paging.c -o paging
vaibhavi@VYSRI:~/os-scheduling$ ./paging
Enter page number (0-3): 3
Page Fault!
Page must be loaded into memory.
vaibhavi@VYSRI:~/os-scheduling$

```

4. Address Translation:

```

GNU nano 8.7.1
#include <stdio.h>

int main() {

    int page_table[] = {3, 5, 1, 7};
    int page_size = 100;

    int virtual_address = 250;

    int page = virtual_address / page_size;
    int offset = virtual_address % page_size;

    int frame = page_table[page];

    int physical_address = frame * page_size + offset;

    printf("Virtual Address = %d\n", virtual_address);
    printf("Page Number      = %d\n", page);
    printf("Offset           = %d\n", offset);
    printf("Frame Number       = %d\n", frame);
    printf("Physical Address = %d\n", physical_address);

    return 0;
}

```

```

vaibhavi@VYSRI:~/os-scheduling$ cd ~/os-scheduling
vaibhavi@VYSRI:~/os-scheduling$ nano address_translation.c
vaibhavi@VYSRI:~/os-scheduling$ gcc address_translation.c -o address_translation
vaibhavi@VYSRI:~/os-scheduling$ ./address_translation
Virtual Address = 250
Page Number      = 2
Offset           = 50
Frame Number     = 1
Physical Address = 150
vaibhavi@VYSRI:~/os-scheduling$

```

5. Malloc and Free

```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>

int main() {

    int *p;

    p = malloc(5 * sizeof(int));

    if (p == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        p[i] = (i + 1) * 10;
    }

    for (int i = 0; i < 5; i++) {
        printf("%d ", p[i]);
    }

    printf("\n");

    free(p);

    return 0;
}

```

```

vaibhavi@VYSRI:~/os-scheduling$ cd ~/os-scheduling
vaibhavi@VYSRI:~/os-scheduling$ nano malloc_free.c
vaibhavi@VYSRI:~/os-scheduling$ ls
address_translation    dynamic_memory    malloc_free.c    memory.c    paging.c    pipes.c    signal.c    virtual_address.c
address_translation.c  dynamic_memory.c  memory           paging      pipes      signal     virtual_address
vaibhavi@VYSRI:~/os-scheduling$ gcc malloc_free.c -o malloc_free
vaibhavi@VYSRI:~/os-scheduling$ ./malloc_free
10 20 30 40 50
vaibhavi@VYSRI:~/os-scheduling$

```

6. Segmentation Fault

```
GNU nano 8.7.1                                segmentation_fault.c *
#include <stdio.h>

int main() {

    int *p = NULL;

    printf("Before accessing memory\n");

    *p = 100;

    printf("After accessing memory\n");

    return 0;
}
```

```
vaibhavi@VYSRI:~/os-scheduling$ cd ~/os-scheduling
vaibhavi@VYSRI:~/os-scheduling$ nano segmentation_fault.c
vaibhavi@VYSRI:~/os-scheduling$ gcc segmentation_fault.c -o segmentation_fault
vaibhavi@VYSRI:~/os-scheduling$ ./segmentation_fault
Before accessing memory
Segmentation fault
vaibhavi@VYSRI:~/os-scheduling$
```